



WEEKLY INTERNSHIP PROGRESS REPORT

Module: Week 2 – Database & CRUD Operations in Laravel

Submitted By: Muneeb ur Rehman

Role: Full-Stack Web Developer Intern

Stack/Focus: PHP, Laravel, MySQL, Bootstrap 5

Report Date: 11-Aug-2026

Submitted To: POF IT Dept

1. Executive Summary

The primary goal of Week 1 was to transition from foundational backend web development concepts to modern, framework-driven architecture using Laravel. The week focused on establishing a local development environment, mastering procedural and Object-Oriented PHP (OOP), and successfully routing a dynamic, multi-page Laravel application using the MVC (Model-View-Controller) design pattern.

2. Key Achievements & Task Breakdown

- **Environmental Architecture:** Deployed local server infrastructure using **XAMPP** (Apache/MySQL) and configured **Composer** for robust PHP dependency management.
- **Core PHP Programming:** Engineered custom PHP scripts utilizing arrays, iterative loops, and complex control structures to establish core backend logic.
- **Object-Oriented PHP (OOP):** Constructed scalable data structures using PHP Classes, implementing inheritance, access modifiers, Traits, and Namespaces to prepare for framework integration.
- **Framework Initialization:** Generated a fresh Laravel project architecture via Composer CLI and configured `.env` application keys and local environment variables.
- **Routing & Controllers:** Programmed standard HTTP `GET` routes in `web.php` and isolated application logic into dedicated Controllers using Artisan commands.
- **Blade Templating:** Engineered dynamic frontend views by passing PHP arrays and variables from backend Controllers directly into Blade templates for structural rendering.

3. Technical Implementation Highlights

- **Dependency Management:** Transitioned to standard modern PHP practices by utilizing Composer, ensuring framework libraries are isolated, version-controlled, and easily updatable.
- **Separation of Concerns:** Strictly adhered to MVC data flow. Routing logic was minimized, offloading processing to Controllers, which injected clean data objects into Blade Views.
- **OOP Scalability:** Utilizing PHP Traits and Namespaces provided the exact architectural understanding required to navigate Laravel's core system files and object-relational mapping efficiently.

4. Conclusion

Week 1 successfully established the critical backend infrastructure required for advanced application development. By mastering core OOP PHP principles and setting up a stable local server environment, the transition into the Laravel framework was seamless. The week concluded with a firm understanding of fundamental MVC routing and dynamic Blade templating, providing a stable foundation for the database integration tasks scheduled for Week 2.